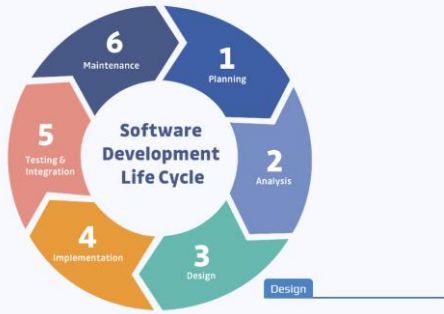




Design:

- Perancangan adalah proses merencanakan atau mempersiapkan suatu rencana atau strategi untuk mencapai tujuan tertentu.
- Ini melibatkan identifikasi masalah atau kebutuhan, pemikiran kreatif, pemilihan solusi terbaik, dan penyusunan rencana tindakan untuk mencapai hasil yang diinginkan.

Tahapan Perancangan Basis Data



a. Identifikasi Kebutuhan Pengguna

- Memahami Kebutuhan bisnis atau aplikasi
- Menentukan User dan Bidang Aplikasi
- Meninjau dokumentasi yang ada
- Menganalisis lingkungan operasi dan pemrosesan data
- Membuat daftar pertanyaan untuk wawancara tentang data apa saja yang diperlukan dan dianalisis

b. Perancangan Konseptual

- Merancang model data konseptual
- Menggambarkan entitas dan relasi secara independen

c. Perancangan Logical

- Terjemahkan model konseptual ke skema basis data yang lebih spesifik
- Pemetaan entitas, atribut, hubungan atau relasi ke dalam tabel, kolom, dan kunci

d. Normalisasi

- Mengorganisasi data dalam tabel
- Menghindari anomali data seperti redundansi, inkonsistensi, dan dependency
- Meningkatkan integritas data dan efisiensi operasional

e. Perancangan Fisik

- Merancang struktur DB sesuai teknologi DB
- Melibatkan tipe data, indeks, partisi, dan strategi penyimpanan

f. Implementasi

Membuat DB sesuai design

g. Uji dan Optimasi

Memastikan keakuratan dan optimisasi kinerja

h. Dokumentasi

Mendokumentasikan struktur DB, skema, aturan integritas, dan proses pengelolaan data

i. Pemeliharaan dan Pengembangan

Melakukan pemeliharaan rutin, pembaruan, dan pengembangan basis data sesuai dengan evolusi kebutuhan bisnis dan teknologi.

Contoh Studi Kasus Toko Elektronik Daring

1. Analisis Kebutuhan

- Toko daring ini memiliki berbagai jenis produk elektronik seperti smartphone, laptop, kamera, dan aksesoris.
- Pelanggan dapat melakukan pembelian produk, melacak pesanan, dan memberikan ulasan tentang produk yang telah mereka beli.
- Manajer toko perlu melacak inventaris produk, mengelola kategori produk, dan menganalisis data penjualan.

2. Perancangan Konseptual

- Identifikasi entitas utama: Produk, Pelanggan, Pesanan, Ulasan, Pegawai Toko.
- Hubungan antara entitas:
 - Pelanggan melakukan Pesanan untuk membeli Produk tertentu
 - Pelanggan memberikan Ulasan tentang Produk.
 - Pegawai melayani pesanan Pelanggan
- Buat diagram ERD yang mencakup entitas dan hubungan antara mereka.

3. Perancangan Logical

- Translasikan model konseptual ke dalam skema tabel: Tabel Produk, Tabel Pelanggan, Tabel Pesanan, Tabel Ulasan, Tabel Pegawai, dll.

- Tentukan atribut untuk setiap tabel.

- Tentukan kunci primer dan kunci asing untuk menjaga integritas referensial.

4. Normalisasi

- Normalisasi setiap tabel untuk mengurangi redundansi dan anomali data.
- Tahapan ini memerlukan data
- Dalam kasus nyata, perlu lakukan pengumpulan data baik berupa faktur, transaksi, maupun proses bisnis.

5. Perancangan Fisik

- Pilih tipe data yang sesuai untuk setiap kolom dalam setiap tabel.

- Buat indeks untuk kolom yang sering diakses atau digunakan dalam pencarian.

- Pertimbangkan strategi penyimpanan dan partisi berdasarkan kebutuhan.

6. Implementasi

- Buat basis data menggunakan sistem manajemen basis data (DBMS) seperti MySQL, PostgreSQL, atau MongoDB.

- Buat tabel-tabel sesuai dengan skema yang telah dirancang.

- Buat kunci primer, kunci asing, dan indeks yang diperlukan.

7. Uji dan Optimasi

- Uji basis data untuk memastikan fungsionalitas yang benar dan kinerja yang memadai.

- Identifikasi dan perbaiki masalah kinerja jika diperlukan dengan melakukan pengindeksan yang tepat atau optimisasi kueri.

8. Dokumentasi

- Dokumentasikan struktur basis data, aturan integritas, dan skema tabel.

- Buat dokumentasi untuk proses pengelolaan data, seperti backup dan pemulihan.

9. Pemeliharaan dan Pengembangan

- Tetap melakukan pemeliharaan rutin, seperti pembaruan skema dan perbaikan bug.

- Perluas basis data sesuai dengan pertumbuhan bisnis atau kebutuhan baru.

LATIHAN

Sistem Manajemen Perpustakaan

- Perpustakaan adalah tempat di mana buku disimpan, dipinjam, dan dikembalikan oleh anggota.
- Analisis Kebutuhan:
 - Sistem perpustakaan harus dapat menyimpan informasi tentang buku, anggota, pinjaman, dan pengembalian.
 - Setiap buku memiliki judul, penulis, tahun terbit, dan nomor identifikasi unik.
 - Anggota memiliki informasi pribadi seperti nama, alamat, nomor telepon, dan nomor identifikasi unik.
 - Setiap pinjaman memiliki informasi tentang buku yang dipinjam, anggota yang meminjam, tanggal pinjam, dan tanggal jatuh tempo.

Nama Anggota	Nomor Anggota	Judul Buku	Tanggal Peminjaman	Tanggal Pengembalian
Ahmad	001	Harry Potter dan Batu Bertuah	2024-02-10	2024-02-17
Siti	002	The Great Gatsby	2024-02-12	2024-02-19
Ahmad	001	Lord of The Rings	2024-02-15	2024-02-22
Budi	003	Harry Potter dan Batu Bertuah	2024-02-16	2024-02-23
Nina	004	To Kill a Mockingbird	2024-02-18	2024-02-25

Lanjutkan hingga ke tahap Uji dan Optimisasi!

REVIEW

- Jelaskan jenis-jenis atribut dan keys pada konsep basis data.
- Jelaskan bentuk-bentuk kardinalitas yang digunakan pada basis data.
- Lakukan normalisasi untuk Faktur Pembelian Bahan Bangunan sebelah!

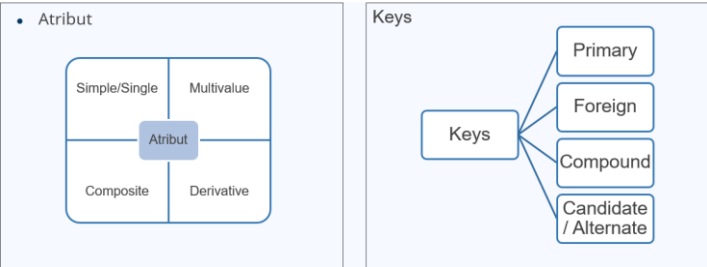
Asumsi:

- Pelanggan atau pembeli tidak perlu disimpan datanya ke database

PD. Harpan Jl. Mendut 20 Tangerang		No. Faktur : 01 Kepada : Aisyah	
FAKTUR			
Banyaknya	Nama Barang	Harga Satuan	Jumlah
10 kaleng	Cat tembok	Rp. 15.000,00	Rp. 150.000,00
10 lembar	Triplek	Rp. 8.000,00	Rp. 80.000,00
		Total	Rp. 230.000,00
Hormat kami, Tangerang, 8 Oktober 2000 Kamila			



NO FAKTUR	KEPADA	TANGGAL	BANYAKNYA	NAMA BARANG	HARGA SATUAN	JUMLAH	TOTAL	PEGAWAI
01	Aisyah	8 Oktober 2000	10 Kaleng	Cat Tembok	15000	150000	230000	Kamila
			10 Lembar	Triplek	8000	80000		



- One-to-One
- One-to-Many / Many-to-One
- Many-to-Many

1 NF

NO FAKTUR	KEPADA	TANGGAL	BANYAKNYA	NAMA BARANG	HARGA SATUAN	JUMLAH	TOTAL	PEGAWAI
1	Aisyah	8 Oktober 2000	10 Kaleng	Cat Tembok	15000	150000	150000	Kamila
1	Aisyah	8 Oktober 2000	10 Lembar	Triplek	8000	80000	80000	Kamila

NO FAKTUR	KEPADA	TANGGAL	BANYAKNYA	KODE BARANG	NAMA BARANG	HARGA SATUAN	JUMLAH	TOTAL	PEGAWAI
1	Aisyah	8 Oktober 2000	10 Kaleng	BRG01	Cat Tembok	15000	150000	150000	Kamila
1	Aisyah	8 Oktober 2000	10 Lembar	BRG02	Triplek	8000	80000	80000	Kamila

1 NF

NO FAKTUR	KEPADA	TANGGAL	BANYAKNYA	KODE BARANG	NAMA BARANG	HARGA SATUAN	JUMLAH	TOTAL	PEGAWAI
1	Aisyah	8 Oktober 2000	10 Kaleng	BRG01	Cat Tembok	15000	150000	150000	Kamila
1	Aisyah	8 Oktober 2000	10 Lembar	BRG02	Triplek	8000	80000	80000	Kamila

PK = No Faktur, Kode barang

Dependency Parsial : No Faktur, Kode Barang → Kepala, Tanggal, Pegawai

Dependency Parsial : No Faktur, Kode Barang → Nama Barang, Harga Satuan

Dependency Fungsional : No Faktur, Kode Barang → Banyaknya, Jumlah, Total

2 NF (ASUMSI 1)

Asumsi 1: Pegawai Pengelola Faktur Tetap atas Nama Kamila

Faktur				Faktur Barang					
NO FAKTUR	KEPADA	TANGGAL	PEGAWAI	NO FAKTUR	KODE BARANG	JUMLAH	TOTAL	BANYAKNYA	
1	Aisyah	8 Oktober 2000	Kamila	1	BRG01	150000	150000	10 Kaleng	
				1	BRG02	80000	80000	10 Lembar	

Barang		
KODE BARANG	NAMA BARANG	HARGA SATUAN
BRG01	Cat Tembok	15000
BRG02	Triplek	8000

2 NF (ASUMSI 2)

Asumsi 2: Pegawai Pengelola Faktur Tidak Hanya Kamila

Faktur					Faktur Barang				
NO FAKTUR	KEPADA	TANGGAL	KODE PEGAWAI	PEGAWAI	NO FAKTUR	KODE BARANG	JUMLAH	TOTAL	BANYAKNYA
1	Aisyah	8 Oktober 2000	PG01	Kamila	1	BRG01	150000	150000	10 Kaleng
					1	BRG02	80000	80000	10 Lembar

Barang		
KODE BARANG	NAMA BARANG	HARGA SATUAN
BRG01	Cat Tembok	15000
BRG02	Triplek	8000

Asumsi 1: Pegawai Pengelola Faktur Tidak Hanya Kamila

3 NF

Faktur				Faktur Barang				
NO FAKTUR	KEPADA	TANGGAL	KODE PEGAWAI	NO FAKTUR	KODE BARANG	JUMLAH	TOTAL	BANYAKNYA
1	Aisyah	8 Oktober 2000	PG01	1	BRG01	150000	150000	10 Kaleng
				1	BRG02	80000	80000	10 Lembar

Pegawai		Barang		
KODE PEGAWAI	PEGAWAI	KODE BARANG	NAMA BARANG	HARGA SATUAN
PG01	Kamila	BRG01	Cat Tembok	15000
		BRG02	Triplek	8000

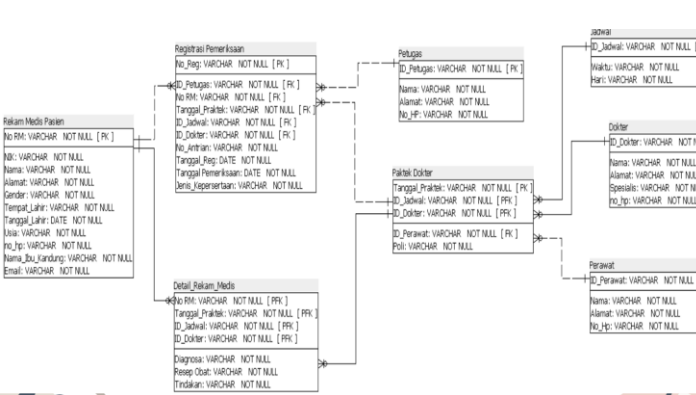
Latihan SIM Rekam MEDIS

INFORMASI

- Sistem harus dapat menyimpan dan mengelola informasi tentang pasien, rekam medis, diagnosa, pengobatan, dan riwayat pemeriksaan, serta tenaga medis yang melayani baik dokter maupun petugas medis dan non-medis lainnya.
- Nomor rekam medis diperoleh dari pendaftaran sebagai pasien di klinik dengan mengisikan data pasien seperti nama, tempat lahir, tanggal lahir, usia, Alamat, gender, no telepon, nama ibu kandung.
- Jika sudah terdaftar maka bisa lanjut ke proses pendaftaran pemeriksaan.
- Proses pendaftaran pemeriksaan pasien didahului dengan proses registrasi untuk mendapatkan antrian.
- Pendaftaran pemeriksaan hanya menyimpan nomor antrian periksa, nama pasien, nomor rekam medis, tanggal pemeriksaan, tanggal registrasi, jadwal, dan dokter yang memeriksa.
- Pendaftaran dapat dilakukan H-3 pemeriksaan.

INFORMASI

- Pasien menunggu dipanggil petugas medis yang mendampingi saat pemeriksaan dengan dokter untuk diperiksa oleh dokter sesuai dengan nomor antrian yang sudah diberikan.
- Petugas yang membantu atau mendampingi dokter dapat berubah sesuai shift.
- Selama pemeriksaan, rekam medis diberikan kepada dokter untuk melihat Riwayat Pemeriksaan.
- Hasil pemeriksaan medis berisikan diagnose yang disimpan dalam rekam medis. Hasil pemeriksaan disimpan berupa tanggal periksa, dokter yang menangani, petugas medis yang melayani, hasil diagnose, dan pengobatannya.
- Pasien bisa saja melakukan pemeriksaan dengan dokter yang berbeda dan diagnose yang berbeda
- Rekam medis menjadi sebuah dokumen Riwayat pemeriksaan dan penanganan dari pasien dan tidak dapat dihapus ataupun diubah.



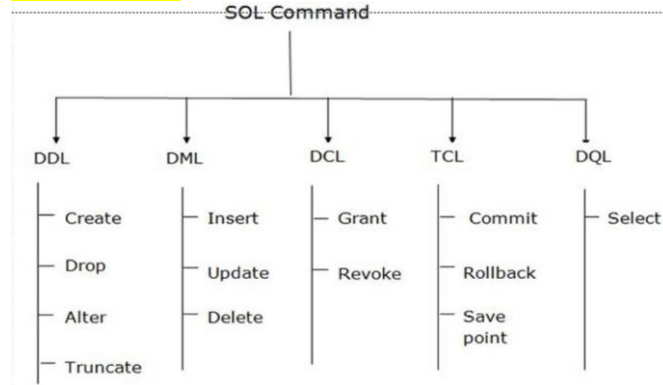
Pengantar SQL (Structured Query Language)

- Bahasa Pemrograman
- Untuk menyimpan dan memproses informasi pada DBMS
- Developed by IBM in early 1970s
- Bersifat tidak case sensitive

Domain Types in SQL

- **char(n).** Fixed length character string, with user-specified length *n*.
- **varchar(n).** Variable length character strings, with user-specified maximum length *n*.
- **int.** Integer (a finite subset of the integers that is machine-dependent).
- **smallint.** Small integer (a machine-dependent subset of the integer domain type).
- **numeric(p,d).** Fixed point number, with user-specified precision of *p* digits, with *d* digits to the right of decimal point. (ex., **numeric(3,1)**, allows 44.5 to be stored exactly, but not 444.5 or 0.32)
- **real, double precision.** Floating point and double precision floating
- point numbers, with machine-dependent precision.
- **float(n).** Floating point number, with user-specified precision of at least *n* digits.
- More are covered in Chapter 4.

Perintah Dasar SQL



Data Definition Language

1. **DDL**
 - Perintah untuk merubah, membuat, menghapus, dll pada tabel
 - Perintah: create, alter, drop, truncate
 - Integrity-Constraint:
 - Primary Key
 - Foreign Key
 - Not Null

CREATE

create table r ($A_1D_1, A_2D_2, \dots, A_nD_n$, **integrity – constraint**)

r = nama tabel

A = nama atribut

D = tipe data

```
create table department
(dept_name varchar (20),
building   varchar (15),
budget    numeric (12,2),
primary key (dept_name));
```

```
create table instructor
(ID          varchar (5),
name        varchar (20) not null,
dept_name   varchar (20),
salary     numeric (8,2),
primary key (ID),
foreign key (dept_name) references department);
```

Membuat tabel

-- Tabel Pengirim

CREATE TABLE Pengirim (

id_Pengirim VARCHAR(5) NOT NULL PRIMARY KEY,

nama VARCHAR(100) NOT NULL,

alamat VARCHAR(100) NOT NULL,

telp INTEGER NOT NULL,

id_kota VARCHAR(10) NOT NULL,

FOREIGN KEY (id_kota) REFERENCES Kota(id_kota) ON DELETE CASCADE
ON UPDATE CASCADE

);

DROP & ALTER

Drop:

drop table r

Alter:

- We use the alter table command to add attributes to an existing relation.
- All tuples in the relation are assigned null as the value for the new attribute.

alter table r add A D

Penggunaan Alter dan Drop

ALTER

1. Menambah Kolom Baru

```
ALTER TABLE produk
ADD stok INT;
```

Sekarang, tabel produk akan memiliki kolom stok baru.

2. Mengubah Tipe Data Kolom

```
ALTER TABLE produk
ALTER COLUMN harga TYPE DECIMAL(10, 2);
```

3. Mengganti Nama Kolom

Untuk mengganti nama kolom nama_produk menjadi nama_barang:

```
ALTER TABLE produk
RENAME COLUMN nama_produk TO nama_barang;
```

4. Menghapus Kolom

Misalnya, kita ingin menghapus kolom stok dari tabel produk:

```
ALTER TABLE produk DROP COLUMN stok;
```

5. Mengganti Nama Tabel

Untuk mengganti nama tabel produk menjadi barang:

```
ALTER TABLE produk RENAME TO barang;
```

DROP

1. Menghapus Tabel

```
DROP TABLE barang;
```

2. Menghapus Kolom

Misalnya kita ingin menghapus kolom stok dari tabel produk:

```
DROP COLUMN stok;
```

2. DML

SQL DML menyediakan kemampuan untuk mengambil informasi dari basis data serta untuk menyisipkan tuple ke dalam, menghapus tuple dari, dan memodifikasi tuple di dalam basis data.

- Perintah:
 - Select
 - Insert
 - Update
 - Delete
 - Create

SELECT

- Bentuk Pertama
 - Memanggil seluruh atribut dari satu tabel
- select * from r**

- Bentuk Kedua
 - Memanggil beberapa atribut tertentu dari satu tabel

select $A_1, A_2, \dots, A_n$ from r

***select name
from instructor;***

r = nama tabel
 A = nama atribut

WHERE

Klausula **WHERE** digunakan untuk memfilter catatan. Klausula ini digunakan untuk mengekstrak hanya catatan yang memenuhi kondisi yang ditentukan.

***select * from r where $A_1 = V_1$
select * from Customers where Country = 'Mexico';***

r = tabel
 a = atribut
 v = value

contoh:
SELECT * FROM Pengirim
SELECT nama FROM Pengirim

UPDATE

Untuk mengubah nilai dalam tuple tanpa mengubah semua nilai dalam tuple

***update r
set $A_n = V$
where P***

r = tabel
 A = Atribut
 V = nilai yang baru
 P = kondisi bernilai *true*

Contoh:

UPDATE mahasiswa SET alamat='Padang' WHERE nim=1110952001

Mahasiswa= nama tabel
Padang = isi yang ingin diubah
Nim = data disebelahnya (meyakinkan nilai mana yang mau diubah)

INSERT

- Untuk menyisipkan data ke dalam sebuah relasi, kita bisa menentukan sebuah tuple yang akan disisipkan atau menulis sebuah query yang hasilnya berupa sekumpulan tuple yang akan disisipkan.
- Jelas bahwa nilai atribut dari tuple yang disisipkan harus merupakan anggota dari domain atribut yang bersangkutan.
- Demikian pula, tuple yang disisipkan harus memiliki jumlah atribut yang sesuai."

***insert into r
values($v_1, v_2, \dots, v_n$)***

***insert into student
values ('3003', 'Green', 'Finance', null);***

	id_kota	nama	provinsi
☐	Edit	Delete	K001 Jakarta DKI Jakarta
☐	Edit	Delete	K002 Bandung Jawa Barat
☐	Edit	Delete	K003 Surabaya Jawa Timur
☐	Edit	Delete	K004 Medan Sumatera Utara
☐	Edit	Delete	K005 Makassar Sulawesi Selatan

Sintaks:

INSERT INTO Kota VALUES
('K001', 'Jakarta', 'DKI Jakarta',
('K002', 'Bandung', 'Jawa Barat',
('K003', 'Surabaya', 'Jawa Timur',
('K004', 'Medan', 'Sumatera Utara'),
('K005', 'Makassar', 'Sulawesi Selatan');

DELETE

Permintaan penghapusan (delete) dinyatakan dengan cara yang hampir sama seperti sebuah query. Kita hanya dapat menghapus seluruh tuple; kita tidak bisa menghapus nilai pada atribut tertentu saja

***delete from r
where P ;***

delete from instructor;

r = tabel
 P = kondisi bernilai *true*

Contoh :

**DELETE FROM mahasiswa
DELETE FROM mahasiswa WHERE nim=1110952001**

Mahasiswa = tabel yang ingin dihapus
Nim = kolom yang ingin dihapus

SQL Intermediate

AGGREGATE FUNCTIONS

Fungsi agregat adalah fungsi yang melakukan perhitungan terhadap sekumpulan nilai, dan mengembalikan satu nilai saja. Fungsi agregat sering digunakan bersama klausa **GROUP BY** dalam pernyataan **SELECT**. Klausula **GROUP BY** membagi hasil query menjadi kelompok-kelompok nilai, dan fungsi agregat dapat digunakan untuk mengembalikan satu nilai untuk setiap kelompok tersebut. Fungsi agregat SQL yang paling umum digunakan adalah:

- MIN()** – mengembalikan nilai terkecil dari kolom yang dipilih
 - MAX()** – mengembalikan nilai terbesar dari kolom yang dipilih
 - COUNT()** – mengembalikan jumlah baris dalam satu set
 - SUM()** – mengembalikan total dari kolom numerik
 - AVG()** – mengembalikan nilai rata-rata dari kolom numerik
- Fungsi agregat **mengabaikan nilai null** (kecuali untuk COUNT()).

1. MIN() & MAX()

- The MIN() function returns the smallest value of the selected column.
- The MAX() function returns the largest value of the selected column.

Example:

- select min(price) from products;**
- select max(price) from products;**

2. COUNT()

The COUNT() function returns the number of rows that matches a specified criterion. (jumlah baris dalam satu set)

Example:

- select count(*) from products;**
- select count(product_name) from products;**
- select count(product_name) from products where price > 20000;**
- Ignore Duplicate
 - select count(distinct product_name) from products;**
- Using alias
 - select count(distinct product_name) as [product name] from products;**

3. SUM()

The SUM() function returns the total sum of a numeric column. (total kolom numerik)

Example:

- select sum(quantity) from OrderDetails;**
- select sum(quantity*10) from OrderDetails;**

4. AVG()

The AVG() function returns the average value of a numeric column. (nilai rata rata)

Example:

- select avg(price) from Products;**
- select * from products where price > (select avg(price) from Products);**

LIKE OPERATOR

Operator LIKE digunakan dalam klausa **WHERE** untuk mencari pola tertentu dalam sebuah kolom.

Terdapat dua wildcard (karakter pengganti) yang sering digunakan bersama operator LIKE:

- Tanda persen % mewakili nol, satu, atau lebih karakter
- Tanda garis bawah _ mewakili satu karakter tunggal

Contoh:

select * from Customers where CustomerName LIKE 'a%';

1. SELECT-WHERE

• Bentuk Ketiga

- Memanggil beberapa atribut dari beberapa tabel

```
select r1.A1, r2.A2
from r1, r2
where r1.A1 = r2.A1
```

```
select name, instructor.dept_name, building
from instructor, department
where instructor.dept_name = department.dept_name;
```

- Example :

```
Select customer.nama_customer, faktur.id_faktur
From customer, faktur
Where customer.id_customer = faktur.id_customer
```

r = nama tabel

A = nama atribut

2. SELECT-AND

• Bentuk Ketiga

- Memanggil beberapa atribut dari beberapa tabel

```
select r1.A1, r2.A2
from r1, r2
where r1.A1 = r2.A1
```

```
select name, instructor.dept_name, building
from instructor, department
where instructor.dept_name = department.dept_name;
```

r = nama tabel

A = nama atribut

JOIN

Klausula **JOIN** digunakan untuk menggabungkan baris dari dua atau lebih tabel, berdasarkan kolom yang saling berhubungan di antara mereka

- Example :

SELECT Orders.OrderID, Customers.CustomerName, Orders.OrderDate

FROM Orders

INNER JOIN Customers

ON Orders.CustomerID=Customers.CustomerID;

Let's look at a selection from the "Orders" table:

OrderID	CustomerID	OrderDate
10308	2	1996-09-18
10309	37	1996-09-19
10310	77	1996-09-20

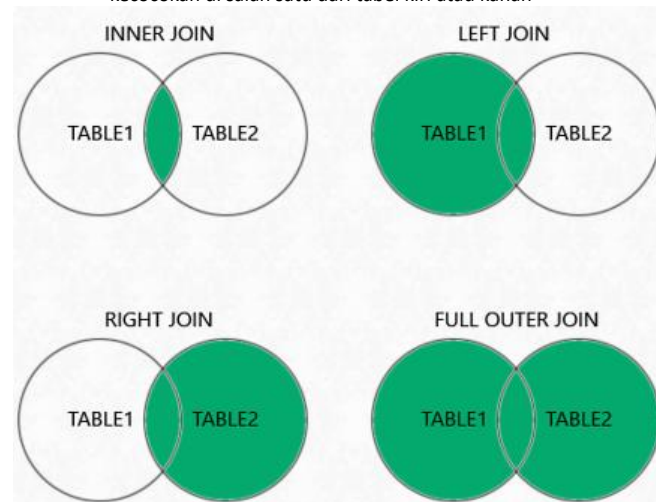
Then, look at a selection from the "Customers" table:

CustomerID	CustomerName	ContactName	Country
1	Alfreds Futterkiste	Maria Anders	Germany
2	Ana Trujillo Emparedados y helados	Ana Trujillo	Mexico
3	Antonio Moreno Taquería	Antonio Moreno	Mexico

TIPE JOIN

Berikut adalah berbagai jenis **JOIN** dalam SQL:

- **(INNER) JOIN**: Mengembalikan data yang memiliki nilai yang cocok di kedua tabel
- **LEFT (OUTER) JOIN**: Mengembalikan semua data dari tabel kiri, dan data yang cocok dari tabel kanan
- **RIGHT (OUTER) JOIN**: Mengembalikan semua data dari tabel kanan, dan data yang cocok dari tabel kiri
- **FULL (OUTER) JOIN**: Mengembalikan semua data ketika terdapat kecocokan di salah satu dari tabel kiri atau kanan



JOIN MORE THAN 2 TABLES

- Setelah melakukan *join* pada tabel pertama dan kedua, kamu bisa menambahkan tabel ketiga dengan cara yang serupa:

```
SELECT kolom1, kolom2
FROM tabel1
INNER JOIN tabel2 ON tabel1.primary_key =
tabel2.foreign_key_tabel1
INNER JOIN tabel3 ON tabel2.primary_key =
tabel3.foreign_key_tabel2
WHERE kondisi;
```

- Pastikan untuk memilih kolom yang tepat untuk *join* dan menuliskan klausa WHERE secara akurat, guna menjaga efisiensi dan keakuratan data.

UTS 2024

1. Basis data merupakan sekumpulan data terstruktur dan sistematis yang dapat diolah dan dimanipulasi sehingga menghasilkan informasi yang relevan. Dalam pembangunan sebuah perangkat lunak, bagaimana peranan basis data dan bagaimana urgensi dari sebuah basis data itu?

Basis data memiliki peranan yang sangat penting dalam pembangunan perangkat lunak karena menjadi tempat penyimpanan utama dari seluruh data yang dibutuhkan oleh sistem. Peranannya meliputi:

1. **Menyimpan Data Secara Terstruktur**
Basis data menyimpan data dalam format yang terstruktur sehingga memudahkan akses, pencarian, dan pengelolaan data.
2. **Mempermudah Pengolahan Informasi**
Data yang tersimpan bisa dengan mudah dimanipulasi menggunakan bahasa query seperti SQL untuk menghasilkan informasi yang relevan dan berguna bagi pengguna.
3. **Menjaga Konsistensi dan Keamanan Data**
Sistem basis data dilengkapi dengan mekanisme untuk menjaga integritas, konsistensi, dan keamanan data.
4. **Mendukung Operasional Sistem Secara Real-Time**
Basis data memungkinkan sistem untuk melakukan proses penyimpanan dan pengambilan data secara cepat, terutama pada aplikasi real-time seperti sistem perbankan atau e-commerce.

Adapun urgensinya adalah:

- Tanpa basis data yang baik, perangkat lunak tidak dapat menyimpan atau mengelola data secara efisien.
- Dapat menyebabkan informasi yang dihasilkan menjadi tidak akurat atau tidak relevan.
- Basis data juga memungkinkan integrasi dengan berbagai komponen lain dalam sistem, seperti antarmuka pengguna, sistem backend, dan API.

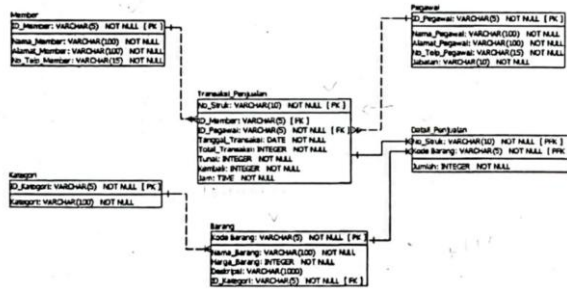
Kesimpulan: Basis data merupakan fondasi utama dalam pengembangan perangkat lunak karena mendukung seluruh proses penyimpanan, pengambilan, dan pengolahan data secara efisien dan aman.

2. Basis data dibuat dengan melalui beberapa tahapan sehingga dapat digunakan secara utuh oleh pengguna. Tuliskan tahapan-tahapan dalam perancangan basis data tersebut satu per satu!

1. Analisis Kebutuhan pengguna
2. Perancangan Konseptual
3. Perancangan Logical
4. Normalisasi
5. Perancangan fisik
6. Implementasi
7. Uji dan optimasi
8. Dokumentasi

9. Pemeliharaan dan pengembangan

3. Perhatikan skema basis data berikut



Skema tersebut merupakan sebuah rancangan basis data penjualan barang di swalayan. Pada gambar terdapat enam tabel yang digunakan. Berikut penjabarannya:

- Member** : Tabel ini menyimpan data member dari Swalayan dengan atribut yang digunakan adalah IDE Member, Nama Member, Alamat Member, No. Telp member.
- Pegawai** : Tabel ini menyimpan data pegawai yang bekerja pada Swalayan dengan atribut yang digunakan ID Pegawai, Nama Pegawai, Alamat Pegawai, No. Telp Pegawai, Jabatan.
- Barang** : Tabel ini menyimpan data barang yang dijual dan dikelola pada Swalayan dengan atribut yang digunakan Kode Barang, Nama Barang, Harga Barang, Deskripsi, ID Kategori.
- Kategori** : Tabel ini menyimpan data kategori barang untuk mengelompokkan barang sesuai kategorinya dengan atribut yang digunakan ID Kategori, Kategori.
- Transaksi Penjualan** : Tabel ini menyimpan data transaksi penjualan pada Swalayan dengan atribut yang digunakan No. Struk, ID Member, IDE Pegawai, Tanggal Transaksi, Total Transaksi, Tunai, Kembali, Jam.
- Detail Penjualan** : Tabel ini menyimpan data detail transaksi penjualan yang memuat data No. Struk, Kode Barang, dan Jumlah.

Berdasarkan keterangan tersebut, kembangkanlah rancangan basis data pada gambar dengan mempertimbangkan informasi berikut

- Adanya proses pengiriman barang yang hanya dapat dilakukan didalam kota yang sama dengan Swalayan.
- Proses pengiriman dilakukan dengan melibatkan Pegawai dan/atau Kurir Pengiriman Instan dalam Kota.
- Buatkan skema final hasil penambahan Poin 1 dan 2.

